
Session 22 (AIML) - Assignment Questions

Name: **Vaishnavi Praveen Rathi**

College: **Zeal Polytechnic, Narhe, Pune**

Branch: **Artificial Intelligence and Machine Learning (AIML)**

Domain: **AIML (Artificial Intelligence and Machine Learning)**

Github Main Repo Link : https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments

Github File Link : https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments/blob/main/Session22/Assignment22.ipynb

Dataset Used: – Kaggle - <https://www.kaggle.com/datasets/fedesoriano/heart-failure-prediction>

GitHub Repository: <https://github.com/vaishnavi-rathiii/Heart-Disease-Streamlit>

Deployed Streamlit App: <https://heart-disease-appgit-w7jxys5qb7jx9ndasge5gl.streamlit.app/>

Q1. Data Loading & Preprocessing

Output Screenshots:

First 5 Rows of Dataset:

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	\
0	40	M	ATA	140	289	0	Normal	172	
1	49	F	NAP	160	180	0	Normal	156	
2	37	M	ATA	130	283	0	ST	98	
3	48	F	ASY	138	214	0	Normal	108	
4	54	M	NAP	150	195	0	Normal	122	

	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	N	0.0	Up	0
1	N	1.0	Flat	1
2	N	0.0	Up	0
3	Y	1.5	Flat	1
4	N	0.0	Up	0

Shape of X: (918, 11)

Shape of y: (918,)

Categorical Columns:

Index(['Sex', 'ChestPainType', 'RestingECG', 'ExerciseAngina', 'ST_Slope'], dtype='object')

First 5 Rows After Encoding:

	Age	RestingBP	Cholesterol	FastingBS	MaxHR	Oldpeak	Sex_F	Sex_M	\
0	40	140	289	0	172	0	0	1	
1	49	160	180	0	156	1	1	0	
2	37	130	283	0	98	0	0	1	
3	48	138	214	0	108	1	1	0	
4	54	150	195	0	122	0	0	1	

	ChestPainType_ASY	ChestPainType_ATA	ChestPainType_NAP	ChestPainType_TA	\
0	0	1	0	0	
1	0	0	1	0	
2	0	1	0	0	
3	1	0	0	0	
4	0	0	1	0	

	RestingECG_LVH	RestingECG_Normal	RestingECG_ST	ExerciseAngina_N	\
0	0	1	0	1	
1	0	1	0	1	
2	0	0	1	1	
3	0	1	0	0	
4	0	1	0	1	

	ExerciseAngina_Y	ST_Slope_Down	ST_Slope_Flat	ST_Slope_Up
0	0	0	0	1
1	0	0	1	0
2	0	0	0	1
3	1	0	1	0
4	0	0	0	1

Program Code:

'''

Q1. (Data Loading & Preprocessing)

Load the Heart Disease dataset.

Separate features into X (independent) and y (dependent/target = HeartDisease).

Perform necessary preprocessing (encoding categorical columns).

'''

```
import pandas as pd
```

```
# Load the dataset
```

```
df = pd.read_csv("heart.csv")
```

```
# Display first 5 rows
```

```
print("First 5 Rows of Dataset:")
```

```
print(df.head())
```

```
# Separate independent and dependent variables
```

```
X = df.drop("HeartDisease", axis=1)
```

```
y = df["HeartDisease"]
```

```
# Display shapes
```

```
print("\nShape of X:", X.shape)
```

```
print("Shape of y:", y.shape)
```

```
# Identify categorical columns
```

```
categorical_columns = X.select_dtypes(include="object").columns
```

```
print("\nCategorical Columns:")
```

```
print(categorical_columns)
```

```
# Perform One-Hot Encoding
```

```
X = pd.get_dummies(X, columns=categorical_columns)
```

```
# Convert encoded columns to integer type
```

```
X = X.astype(int)
```

```
# Display first 5 rows after encoding
```

```
print("\nFirst 5 Rows After Encoding:")
```

```
print(X.head())
```

```
from sklearn.preprocessing import StandardScaler
```

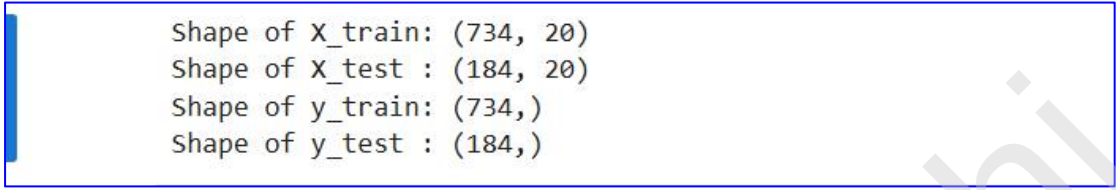
```
# Create StandardScaler object  
scaler = StandardScaler()
```

```
# Scale training and testing data  
X_train = scaler.fit_transform(X_train)  
X_test = scaler.transform(X_test)
```

Vaishnavi Rathni

Q2. Train-Test Split

Output Screenshot:



```
Shape of X_train: (734, 20)
Shape of X_test : (184, 20)
Shape of y_train: (734,)
Shape of y_test : (184,)
```

Program Code:

```
'''
Q2. (Train-Test Split)

Split the preprocessed data into training and testing sets using train_test_split.
Use test_size=0.20 and random_state=42.
Print the shape of X_train, X_test, y_train, and y_test.
'''

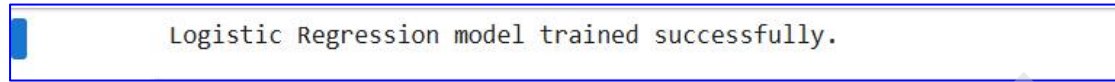
from sklearn.model_selection import train_test_split

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

# Display the shapes of training and testing sets
print("Shape of X_train:", X_train.shape)
print("Shape of X_test :", X_test.shape)
print("Shape of y_train:", y_train.shape)
print("Shape of y_test :", y_test.shape)
```

Q3. Building Logistic Regression Model

Output Screenshot:

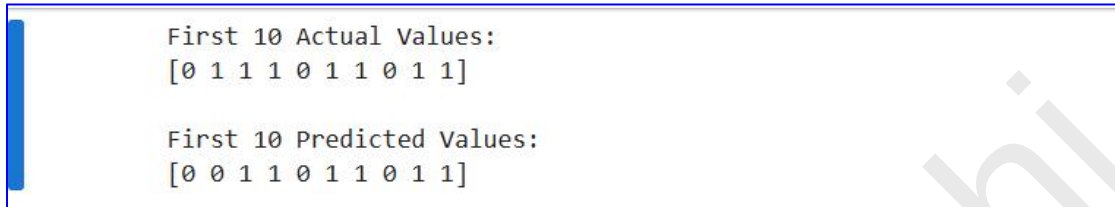
A screenshot of a terminal window with a blue border. Inside, the text "Logistic Regression model trained successfully." is displayed in a monospaced font.

Program Code:

```
'''  
Q3. (Building Logistic Regression Model)  
  
Train a Logistic Regression model:  
Import LogisticRegression from sklearn.  
Create the model and fit it on the training data (X_train, y_train).  
'''  
  
from sklearn.linear_model import LogisticRegression  
  
# Create the Logistic Regression model  
model = LogisticRegression(max_iter=5000)  
  
# Train (fit) the model  
model.fit(X_train, y_train)  
  
print("Logistic Regression model trained successfully.")
```

Q4. Making Predictions

Output Screenshot:

A screenshot of a terminal window with a blue vertical bar on the left. The text inside shows the first 10 actual values and the first 10 predicted values.

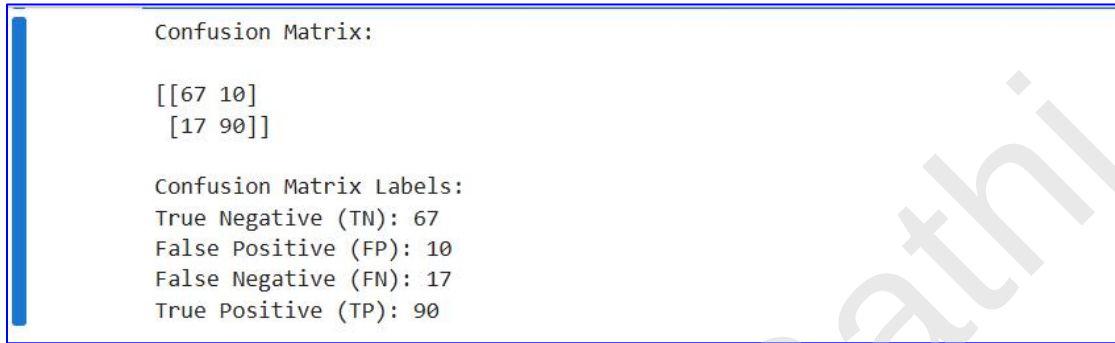
```
First 10 Actual Values:  
[0 1 1 1 0 1 1 0 1 1]  
  
First 10 Predicted Values:  
[0 0 1 1 0 1 1 0 1 1]
```

Program Code:

```
'''  
Q4. (Making Predictions)  
  
Use the trained model to predict on the test data.  
Store predictions in y_pred.  
Print the first 10 actual values (y_test) and predicted values (y_pred).  
'''  
  
# Make predictions on the test data  
y_pred = model.predict(X_test)  
  
# Display first 10 actual values  
print("First 10 Actual Values:")  
print(y_test.iloc[:10].values)  
  
# Display first 10 predicted values  
print("\nFirst 10 Predicted Values:")  
print(y_pred[:10])
```

Q5. Confusion Matrix

Output Screenshot:



```
Confusion Matrix:

[[67 10]
 [17 90]]

Confusion Matrix Labels:
True Negative (TN): 67
False Positive (FP): 10
False Negative (FN): 17
True Positive (TP): 90
```

Program Code:

```
'''
Q5. (Confusion Matrix)

Create a Confusion Matrix using confusion_matrix(y_test, y_pred).
Display the matrix.
Label TN, FP, FN, and TP in the output.
'''

from sklearn.metrics import confusion_matrix

# Create Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

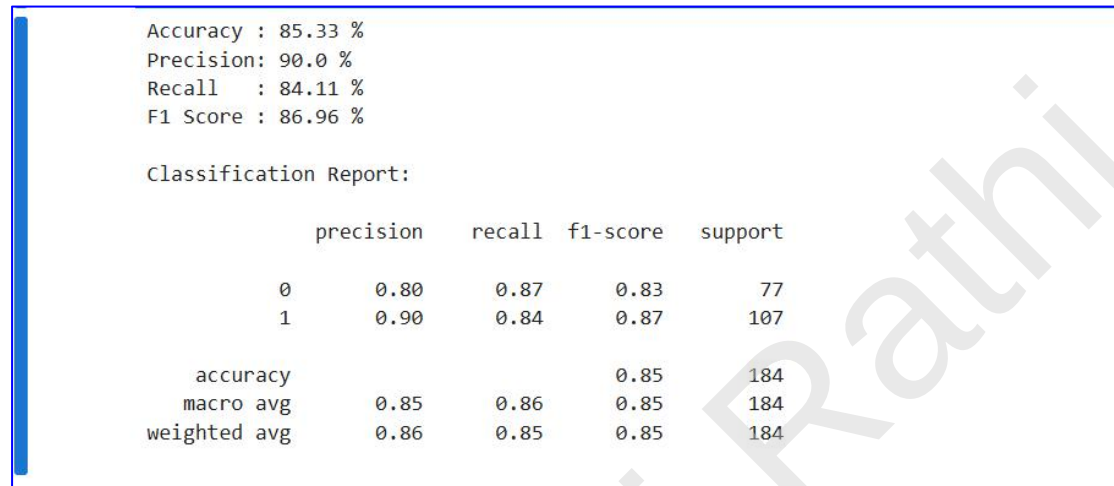
# Display Confusion Matrix
print("Confusion Matrix:\n")
print(cm)

# Extract TN, FP, FN, TP
TN, FP, FN, TP = cm.ravel()

print("\nConfusion Matrix Labels:")
print("True Negative (TN):", TN)
print("False Positive (FP):", FP)
print("False Negative (FN):", FN)
print("True Positive (TP):", TP)
```

Q6. Model Evaluation Metrics

Output Screenshot:

A screenshot of a terminal window showing model evaluation metrics. The output includes Accuracy, Precision, Recall, and F1 Score, followed by a Classification Report table. A large, light gray watermark 'Vaishnavi Rathni' is diagonally across the image.

```
Accuracy : 85.33 %
Precision: 90.0 %
Recall   : 84.11 %
F1 Score : 86.96 %

Classification Report:

              precision    recall  f1-score   support

     0       0.80         0.87         0.83         77
     1       0.90         0.84         0.87        107

 accuracy          0.85          0.85          0.85         184
  macro avg         0.85          0.86          0.85         184
 weighted avg         0.86          0.85          0.85         184
```

Program Code:

'''

Q6. (Model Evaluation Metrics)

Calculate the following metrics:

- Accuracy
- Precision
- Recall
- F1 Score

Use classification_report(y_test, y_pred) and print the full report.

'''

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
classification_report
```

```
# Calculate Evaluation Metrics
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
precision = precision_score(y_test, y_pred)
```

```
recall = recall_score(y_test, y_pred)
```

```
f1 = f1_score(y_test, y_pred)
```

```
# Print Evaluation Metrics
```

```
print("Accuracy :", round(accuracy * 100, 2), "%")
```

```
print("Precision:", round(precision * 100, 2), "%")
```

```
print("Recall   :", round(recall * 100, 2), "%")
```

```
print("F1 Score :", round(f1 * 100, 2), "%")
```

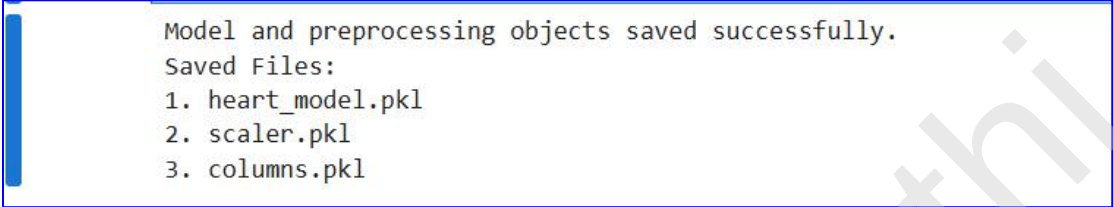
```
# Print Classification Report
```

```
print("\nClassification Report:\n")
```

```
print(classification_report(y_test, y_pred))
```

Q7. Saving the Model

Output Screenshot:



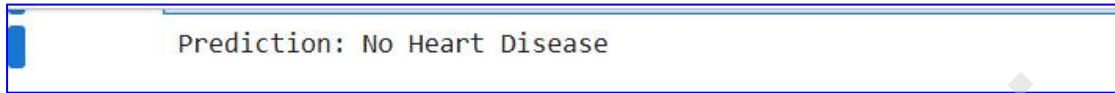
```
Model and preprocessing objects saved successfully.  
Saved Files:  
1. heart_model.pkl  
2. scaler.pkl  
3. columns.pkl
```

Program Code:

```
'''  
Q7. (Saving the Model)  
  
Save the trained model and preprocessing objects using joblib:  
Model → "heart_model.pkl"  
Scaler (if used) → "scaler.pkl"  
Column list → "columns.pkl"  
'''  
  
import joblib  
  
# Save the trained Logistic Regression model  
joblib.dump(model, "heart_model.pkl")  
  
# Save the StandardScaler (only if scaling was used)  
joblib.dump(scaler, "scaler.pkl")  
  
# Save the feature/column names  
joblib.dump(X.columns.tolist(), "columns.pkl")  
  
print("Model and preprocessing objects saved successfully.")  
print("Saved Files:")  
print("1. heart_model.pkl")  
print("2. scaler.pkl")  
print("3. columns.pkl")
```

Q8. Loading & Testing Saved Model)

Output Screenshot:

A screenshot of a prediction output box. The box has a blue border and a blue header bar. The text inside the box reads "Prediction: No Heart Disease".

Prediction: No Heart Disease

Program Code:

'''

Q8. (Loading & Testing Saved Model)

Load the saved model and preprocessing objects.

Create sample input data (one patient record).

Preprocess the input (encoding + scaling).

Make a prediction using the loaded model and print the result.

'''

```
import pandas as pd
import joblib
```

```
# Load saved objects
```

```
model = joblib.load("heart_model.pkl")
```

```
scaler = joblib.load("scaler.pkl")
```

```
encoded_columns = joblib.load("columns.pkl")
```

```
# Create sample patient data
```

```
sample = pd.DataFrame({
```

```
    "Age": [45],
```

```
    "Sex": ["M"],
```

```
    "ChestPainType": ["ATA"],
```

```
    "RestingBP": [130],
```

```
    "Cholesterol": [230],
```

```
    "FastingBS": [0],
```

```
    "RestingECG": ["Normal"],
```

```
    "MaxHR": [150],
```

```
    "ExerciseAngina": ["N"],
```

```
    "Oldpeak": [1.2],
```

```
    "ST_Slope": ["Up"]
```

```
})
```

```
# Perform One-Hot Encoding
```

```
sample = pd.get_dummies(sample)
```

```
# Align columns with training data
```

```
sample = sample.reindex(columns=encoded_columns, fill_value=0)
```

```
# Apply Feature Scaling
```

```
sample_scaled = scaler.transform(sample)
```

```
# Convert scaled data to DataFrame
```

```
sample_scaled = pd.DataFrame(sample_scaled, columns=encoded_columns)
```

```
# Make Prediction
prediction = model.predict(sample_scaled)

# Display Result
if prediction[0] == 1:
    print("Prediction: Heart Disease Detected")
else:
    print("Prediction: No Heart Disease")
```

Q9. (Streamlit Input Interface) & Q10. (Complete Streamlit Web App)

Output Screenshot:

Heart Disease Prediction

Enter the patient details below to predict the presence of heart disease.

Age: 54

Sex: M

Chest Pain Type: ATA

Resting Blood Pressure: 120

Cholesterol: 200

Fasting Blood Sugar: 0

Resting ECG: Normal

Maximum Heart Rate: 150

Exercise Angina: Y

Oldpeak: 1.00

ST Slope: Up

Predict

Heart Disease: No

Program Code:

'''

Q9. (Streamlit Input Interface) & Q10. (Complete Streamlit Web App)

Complete the Streamlit app for Heart Disease Prediction:

- Load the saved model and preprocessing objects.
- On button click, preprocess input and show prediction.

'''

Import Required Libraries

import streamlit as st

import pandas as pd

import joblib

Load Saved Model and Objects

model = joblib.load("heart_model.pkl")

scaler = joblib.load("scaler.pkl")

encoded_columns = joblib.load("columns.pkl")

Page Configuration

st.set_page_config(
 page_title="Heart Disease Prediction",
 layout="centered"
)

st.title("❤️ Heart Disease Prediction")

st.write("Enter the patient details below to predict the presence of heart disease.")

st.divider()

User Inputs

age = st.number_input("Age", 1, 120, 45)

sex = st.selectbox(
 "Sex",
 ["M", "F"]


```

)
chest_pain = st.selectbox(
    "Chest Pain Type",
    ["ATA", "NAP", "ASY", "TA"]
)

resting_bp = st.number_input(
    "Resting Blood Pressure",
    50, 250, 120
)

cholesterol = st.number_input(
    "Cholesterol",
    0, 700, 200
)

fasting_bs = st.selectbox(
    "Fasting Blood Sugar",
    [0, 1]
)

resting_ecg = st.selectbox(
    "Resting ECG",
    ["Normal", "ST", "LVH"]
)

max_hr = st.number_input(
    "Maximum Heart Rate",
    50, 250, 150
)

exercise_angina = st.selectbox(
    "Exercise Angina",
    ["Y", "N"]
)

oldpeak = st.number_input(
    "Oldpeak",
    0.0, 10.0, 1.0
)

st_slope = st.selectbox(
    "ST Slope",
    ["Up", "Flat", "Down"]
)

# -----
# Predict Button
# -----

```

```

if st.button("Predict"):

    try:

        # Create Input DataFrame
        sample = pd.DataFrame({
            "Age": [age],
            "Sex": [sex],
            "ChestPainType": [chest_pain],
            "RestingBP": [resting_bp],
            "Cholesterol": [cholesterol],
            "FastingBS": [fasting_bs],
            "RestingECG": [resting_ecg],
            "MaxHR": [max_hr],
            "ExerciseAngina": [exercise_angina],
            "Oldpeak": [oldpeak],
            "ST_Slope": [st_slope]
        })

        # One-Hot Encoding
        sample = pd.get_dummies(sample)

        # Match Training Columns
        sample = sample.reindex(
            columns=encoded_columns,
            fill_value=0
        )

        # Feature Scaling
        sample_scaled = scaler.transform(sample)

        # Convert back to DataFrame
        sample_scaled = pd.DataFrame(
            sample_scaled,
            columns=encoded_columns
        )

        # Prediction
        prediction = model.predict(sample_scaled)

        # Display Result
        if prediction[0] == 1:
            st.error("❤️ Heart Disease: Yes")
        else:
            st.success("💚 Heart Disease: No")

    except Exception as e:
        st.error("An error occurred while making the prediction.")
        st.exception(e)

```
